

# AttireAI Development Environment Setup Guide

Team 11 - CS407

Team Members:

- Yuanfei Song ([song748@purdue.edu](mailto:song748@purdue.edu))
  - Yichen Dai ([dai229@purdue.edu](mailto:dai229@purdue.edu))
  - Congtian Wu ([wu1908@purdue.edu](mailto:wu1908@purdue.edu))
  - Ekaterina Tsziao ([etszyao@purdue.edu](mailto:etszyao@purdue.edu))
- 

## Table of Contents

1. [System Requirements](#)
  2. [Software Dependencies](#)
  3. [Development Tools](#)
  4. [Installation Instructions](#)
  5. [Project Setup](#)
  6. [Environment Configuration](#)
  7. [Database Setup](#)
  8. [Build and Run Instructions](#)
  9. [Testing](#)
  10. [Debugging and Troubleshooting](#)
  11. [Deployment](#)
- 

## 1. System Requirements

### Minimum Hardware Specifications

- **CPU:** Dual-core processor (2.0 GHz or higher)
- **RAM:** 8 GB minimum (16 GB recommended)
- **Storage:** 5 GB free disk space
- **Internet Connection:** Required for downloading dependencies and Firebase authentication

### Supported Operating Systems

- **macOS:** 10.15 (Catalina) or later
  - **Windows:** 10 or later (Windows 11 recommended)
  - **Linux:** Ubuntu 20.04 LTS or later, Fedora 34+, or equivalent distributions
- 

## 2. Software Dependencies

### Core Technologies

#### Frontend

- **Node.js:** Version 18.17.0 or higher (LTS recommended)
- **npm:** Version 9.6.7 or higher (comes with Node.js)
- **Next.js:** Version 14.x
- **React:** Version 18.x
- **TypeScript:** Version 5.x

#### Backend

- **Python:** Version 3.10 or higher (3.11 recommended)
- **pip:** Version 23.0 or higher
- **FastAPI:** Latest stable version
- **Uvicorn:** ASGI server for FastAPI

#### Database

- **MySQL:** Version 8.0 or higher
- **MySQL Workbench:** Optional but recommended for database management

### Frontend Libraries and Frameworks

```
- next: ^14.x  
  
- react: ^18.x  
  
- react-dom: ^18.x  
  
- typescript: ^5.x  
  
- tailwindcss: ^3.x
```

- `firebase: ^10.x` (for authentication)
- `axios`: For API requests
- `@headlessui/react`: For UI components
- `clsx`: For conditional `className` handling

## Backend Libraries and Frameworks

- `fastapi`: Latest stable
- `uvicorn[standard]`: ASGI server
- `pydantic`: Data validation
- `python-multipart`: File upload support
- `sqlalchemy`: ORM for database
- `pymysql`: MySQL database driver
- `python-dotenv`: Environment variable management
- `firebase-admin`: Firebase authentication on backend
- `python-jose[cryptography]`: JWT token handling
- `passlib[bcrypt]`: Password hashing

---

## 3. Development Tools

### Recommended IDEs and Code Editors

- **VS Code** (Recommended): Lightweight and excellent TypeScript/Python support
- **WebStorm**: Full-featured IDE for JavaScript/TypeScript
- **PyCharm**: Excellent for Python backend development
- **Cursor**: AI-powered code editor (alternative to VS Code)

# Essential VS Code Extensions

- ESLint: JavaScript/TypeScript linting
- Prettier: Code formatting
- Tailwind CSS IntelliSense: Tailwind class autocomplete
- Python: Python language support
- Pylance: Python type checking
- MySQL: Database management within VS Code
- GitLens: Git integration
- Thunder Client or REST Client: API testing

## Additional Tools

- **Git:** Version 2.30 or higher for version control
- **Postman** or **Insomnia:** API testing and development
- **MySQL Workbench:** Database management GUI
- **Firebase Console:** For authentication management (web-based)
- **Chrome DevTools:** For frontend debugging

---

## 4. Installation Instructions

### Step 1: Install Node.js and npm

#### macOS

```
# Using Homebrew (recommended)

brew install node

# Verify installation
```

```
node --version # Should show v18.17.0 or higher
npm --version # Should show v9.6.7 or higher
```

## Windows

1. Download the installer from <https://nodejs.org/>
2. Run the .msi installer
3. Follow the installation wizard (keep default settings)
4. Open Command Prompt and verify:

```
node --version
npm --version
```

## Linux (Ubuntu/Debian)

```
# Using NodeSource repository

curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs

# Verify installation

node --version
npm --version
```

## Step 2: Install Python

### macOS

```
# Using Homebrew
brew install python@3.11

# Verify installation
python3 --version # Should show Python 3.11.x
pip3 --version
```

## Windows

1. Download Python installer from <https://www.python.org/downloads/>

2. **IMPORTANT:** Check "Add Python to PATH" during installation
3. Run the installer
4. Verify in Command Prompt:

```
python --version  
pip --version
```

## Linux (Ubuntu/Debian)

```
sudo apt update  
sudo apt install python3.11 python3.11-venv python3-pip  
  
# Verify installation  
python3 --version  
pip3 --version
```

## Step 3: Install MySQL

### macOS

```
# Using Homebrew  
  
brew install mysql@8.0  
brew services start mysql@8.0  
  
# Secure the installation  
mysql_secure_installation
```

### Windows

1. Download MySQL Installer from <https://dev.mysql.com/downloads/installer/>
2. Choose "Developer Default" setup type
3. Follow the wizard and note your root password
4. Ensure MySQL Server is running in Services

## Linux (Ubuntu/Debian)

```
sudo apt update  
sudo apt install mysql-server
```

```
# Start MySQL service

sudo systemctl start mysql
sudo systemctl enable mysql

# Secure installation

sudo mysql_secure_installation
```

## Step 4: Install Git

### macOS

```
# Git comes pre-installed, but you can update via Homebrew

brew install git
```

### Windows

1. Download from <https://git-scm.com/download/win>
2. Run installer with default settings
3. Choose "Git from the command line and also from 3rd-party software"

### Linux

```
sudo apt install git
```

Verify Git installation:

```
git --version
```

---

## 5. Project Setup

### Step 1: Clone the Repository

```
# Navigate to your desired directory

cd ~/Documents # or C:\Users\YourName\Documents on Windows
```

```
# Clone the repository

git clone <repository-url>
cd AttireAI
```

**Troubleshooting:** If you get a permission error, ensure you have access to the repository or use SSH keys.

## Step 2: Project Structure Overview

```
AttireAI/

├─ backend/ # FastAPI backend
|
| └─ app/
|   |
|   | └─ routers/ # API route handlers
|   |
|   | └─ models/ # Database models
|   |
|   | └─ db/ # Database configuration
|   |
|   | └─ utils/ # Utility functions
|   |
|   └─ requirements.txt # Python dependencies
|
| └─ main.py # FastAPI entry point
├─ src/ # Next.js frontend
|
| └─ app/ # App router pages
|
| └─ components/ # React components
|
| └─ lib/ # Utility libraries
├─ public/ # Static assets
├─ docs/ # Documentation
└─ package.json # Frontend dependencies
```



```
└─ .env.local # Environment variables (create this)
```

### Step 3: Install Frontend Dependencies

```
# From the project root directory
```

```
npm install
```

**Expected output:** You should see a progress bar and `added XXX packages` message.

**If you encounter errors:**

```
# Clear npm cache and retry
```

```
npm cache clean --force
```

```
rm -rf node_modules package-lock.json
```

```
npm install
```

### Step 4: Install Backend Dependencies

```
# Navigate to backend directory
```

```
cd backend
```

```
# Create a Python virtual environment (recommended)
```

```
python3 -m venv venv
```

```
# Activate the virtual environment
```

```
# On macOS/Linux:
```

```
source venv/bin/activate
```

```
# On Windows:
```

```
.\venv\Scripts\activate
```

```
# Your terminal should now show (venv) at the beginning
```

```
# Install dependencies
```

```
pip install -r requirements.txt
```

**Verification:** Run `pip list` to see all installed packages.

---

## 6. Environment Configuration

### Step 1: Frontend Environment Variables

Create a file named `.env.local` in the **project root directory**:

```
# From project root

touch .env.local # macOS/Linux

# Or create manually on Windows
```

Add the following content to `.env.local`:

```
# Firebase Configuration

NEXT_PUBLIC_FIREBASE_API_KEY=your_firebase_api_key_here

NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=your_project_id.firebaseio.com

NEXT_PUBLIC_FIREBASE_PROJECT_ID=your_project_id

NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=your_project_id.appspot.com

NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=your_sender_id

NEXT_PUBLIC_FIREBASE_APP_ID=your_app_id

# API Configuration

NEXT_PUBLIC_API_URL=http://localhost:8000
```

#### How to get Firebase credentials:

1. Go to <https://console.firebase.google.com/>
2. Create a new project or select existing "AttireAI" project
3. Navigate to Project Settings > General
4. Scroll to "Your apps" and select Web app
5. Copy the configuration values

## Step 2: Backend Environment Variables

Create a file named `.env` in the **backend** directory:

```
cd backend
touch .env # macOS/Linux
```

Add the following content to `backend/.env` :

```
# Database Configuration

DATABASE_URL=mysql+pymysql://root:your_password@localhost:3306/attireai_db

DB_HOST=localhost

DB_PORT=3306

DB_USER=root

DB_PASSWORD=your_mysql_root_password

DB_NAME=attireai_db


# Firebase Admin SDK (for backend authentication)

FIREBASE_PROJECT_ID=your_project_id

FIREBASE_PRIVATE_KEY_ID=your_private_key_id

FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\nYour_private_key_here\n-----END PRIVATE KEY-----\n"

FIREBASE_CLIENT_EMAIL=firebase-adminsdk@your_project_id.iam.gserviceaccount.com


# CORS Settings

CORS_ORIGINS=["http://localhost:3000"]
```

```
# Optional: Secret Keys
```

```
SECRET_KEY=generate_a_random_secret_key_here
```

### How to get Firebase Admin SDK credentials:

1. Go to Firebase Console > Project Settings
2. Navigate to "Service accounts" tab
3. Click "Generate new private key"
4. Download the JSON file
5. Extract values from the JSON into your `.env` file

**Generate SECRET\_KEY (optional):**

```
python3 -c "import secrets; print(secrets.token_urlsafe(32))"
```

## Step 3: Verify Configuration Files

**Important:** Add these files to `.gitignore` to prevent committing secrets:

```
# Check if .gitignore exists
cat .gitignore
# Ensure these lines are present:
.env.local
.env
backend/.env
backend/venv/
node_modules
```

---

## 7. Database Setup

### Step 1: Access MySQL

```
# Open MySQL shell
mysql -u root -p
# Enter your MySQL root password when prompted
```

### Step 2: Create Database

```
-- Create the database
CREATE DATABASE attireai_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
-- Verify database was created
SHOW DATABASES;
-- Exit MySQL shell
EXIT;
```

## Step 3: Run Database Migrations

```
# Navigate to backend directory
cd backend
# Ensure virtual environment is activated
source venv/bin/activate # macOS/Linux
# or
.\venv\Scripts\activate # Windows
# Run migration scripts (if available)
# Check if migrations directory exists
ls migrations/
# Apply migrations manually or via script
mysql -u root -p attireai_db < migrations/001_initial_schema.sql
mysql -u root -p attireai_db <
migrations/002_update_color_profiles_fixed.sql
```

## Step 4: Verify Database Schema

```
mysql -u root -p attireai_db
```

```
-- Check tables were created
SHOW TABLES;
-- Expected tables:

-- users

-- measurement_profiles

-- color_profiles

-- style_preferences

-- outfit_recommendations

-- Describe a table to verify structure
```

```
DESCRIBE users;
DESCRIBE color_profiles;

EXIT;
```

### Troubleshooting:

- If migrations fail, check the `.env` file for correct database credentials
  - Ensure MySQL service is running: `brew services list` (macOS) or `services.msc` (Windows)
- 

## 8. Build and Run Instructions

### Running the Development Environment

You'll need **two terminal windows** - one for frontend, one for backend.

#### Terminal 1: Start Backend Server

```
# Navigate to backend directory
cd backend
# Activate virtual environment
source venv/bin/activate # macOS/Linux
.\venv\Scripts\activate # Windows

# Start FastAPI server with hot reload
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

#### Expected output:

```
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)

INFO: Started reloader process

INFO: Started server process

INFO: Waiting for application startup.

INFO: Application startup complete.
```

### Access backend:

- API: <http://localhost:8000>
- API Documentation: <http://localhost:8000/docs> (Swagger UI)
- Alternative Docs: <http://localhost:8000/redoc>

## Terminal 2: Start Frontend Development Server

```
# Navigate to project root (from backend directory)
cd ..
# Start Next.js development server
npm run dev
```

### Expected output:

```
> attireai@0.1.0 dev
> next dev

▲ Next.js 14.x.x
- Local: http://localhost:3000
- Network: http://192.168.x.x:3000

✓ Ready in 2.5s
```

### Access frontend:

- Application: <http://localhost:3000>

## Building for Production

### Frontend Production Build

```
# Create optimized production build
npm run build
```

```
# Preview production build locally
npm run start
```

## Backend Production

```
cd backend
source venv/bin/activate
# Run with production server
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
```

---

## 9. Testing

### Running Frontend Tests (if configured)

```
# Unit tests
npm run test
# End-to-end tests (if configured)
npm run test:e2e
```

### Testing API Endpoints

#### Using Swagger UI (Recommended)

1. Navigate to <http://localhost:8000/docs>
2. Expand any endpoint
3. Click "Try it out"
4. Enter parameters
5. Click "Execute"

#### Using curl

```
# Test health check
curl http://localhost:8000/
# Test color profiles endpoint (requires authentication)
curl -X GET "http://localhost:8000/color-profiles" \
-H "X-Firebase-UID: test_user_id"
# Create color profile
curl -X POST "http://localhost:8000/color-profiles" \
```



```
-H "Content-Type: application/json" \  
-H "X-Firebase-UID: test_user_id" \  
-d '{  
  "skin_tone": "Medium",  
  "skin_tone_hex": "#E8B999",  
  "hair_color": "Black",  
  "hair_color_hex": "#1A1A1A"  
}'
```

## Manual Testing Workflow

### 1. Test Authentication:

- Go to <http://localhost:3000>
- Sign up with email/password
- Verify email confirmation (check Firebase Console)
- Log in

### 2. Test Color Analysis:

- Navigate to color analysis page
- Select skin tone and hair color
- Save profile
- Verify in color results page

### 3. Test Recommendations:

- Select occasion, weather, dress code
- Generate recommendations
- Verify outfit display

---

## 10. Debugging and Troubleshooting

### Common Issues and Solutions

#### Frontend Issues

**Issue:** `npm install` fails

```
# Solution 1: Clear cache  
npm cache clean --force
```

```
rm -rf node_modules package-lock.json
npm install
# Solution 2: Update npm
npm install -g npm@latest
# Solution 3: Use legacy peer deps (if dependency conflicts)
npm install --legacy-peer-deps
```

### Issue: "Port 3000 already in use"

```
# Find and kill process using port 3000
# macOS/Linux:
lsof -ti:3000 | xargs kill -9
# Windows:
netstat -ano | findstr :3000
taskkill /PID <PID_number> /F
# Or use a different port
PORT=3001 npm run dev
```

### Issue: Environment variables not loading

- Restart the development server after changing `.env.local`
- Verify filename is exactly `.env.local` (not `.env.local.txt`)
- Ensure variables start with `NEXT_PUBLIC_` for client-side access

### Issue: Firebase authentication errors

```
# Check Firebase configuration in .env.local

# Verify Firebase project is active in console

# Check browser console for detailed error messages

# Ensure authentication providers are enabled in Firebase Console
```

## Backend Issues

### Issue: `ModuleNotFoundError` when starting backend

```
# Ensure virtual environment is activated
source venv/bin/activate # You should see (venv) in terminal
```

```
# Reinstall requirements
pip install -r requirements.txt

# If specific module missing:
pip install <module_name>
```

### Issue: Database connection fails

```
# Verify MySQL is running
# macOS:
brew services list | grep mysql
# Linux:
sudo systemctl status mysql
# Windows: Check Services app
# Test connection manually
mysql -u root -p -h localhost -P 3306
# Check .env credentials match MySQL user
# Verify database exists:
mysql -u root -p -e "SHOW DATABASES;"
```

### Issue: "Port 8000 already in use"

```
# Find and kill process

# macOS/Linux:

lsof -ti:8000 | xargs kill -9

# Windows:

netstat -ano | findstr :8000

taskkill /PID <PID_number> /F

# Or specify different port
```

```
uvicorn main:app --reload --port 8001
```

### Issue: CORS errors

- Check `CORS_ORIGINS` in `backend/.env`
- Ensure frontend URL is in allowed origins
- Verify CORS middleware is configured in `main.py`

## Database Issues

### Issue: "Access denied for user 'root'@'localhost'"

```
# Reset MySQL root password

# Stop MySQL service first


# macOS:

brew services stop mysql

mysqld_safe --skip-grant-tables &

mysql -u root

# Then in MySQL:

FLUSH PRIVILEGES;

ALTER USER 'root'@'localhost' IDENTIFIED BY 'new_password';

EXIT;

# Restart MySQL normally
```

### Issue: Migration fails

```
# Check migration file syntax
```

```
# Apply migrations one by one

mysql -u root -p attireai_db < migrations/001_initial_schema.sql


# Check for errors in output

# Verify table creation:

mysql -u root -p -e "USE attireai_db; SHOW TABLES;"
```

## Debugging Tools

### Frontend Debugging:

```
// Use console.log in development

console.log('Debug data:', data);


// React DevTools (Chrome extension)

// Inspect component props and state


// Next.js debug mode

DEBUG=* npm run dev
```

### Backend Debugging:

```
# Add print statements

print(f"Debug: {variable}")


# Use FastAPI built-in logging
```

```
import logging

logger = logging.getLogger(__name__)

logger.info("Debug message")


# Use Python debugger

import pdb; pdb.set_trace()
```

### Check Logs:

```
# Backend logs appear in terminal running uvicorn

# Frontend logs in browser console (F12)

# Next.js server logs in terminal running npm run dev
```

---

## 11. Deployment

### Prerequisites for Deployment

- Production database (MySQL on cloud provider)
- Firebase project configured for production
- Domain name (optional but recommended)
- Hosting provider account (Vercel, AWS, Railway, etc.)

### Frontend Deployment (Vercel - Recommended for Next.js)

#### 1. Prepare for deployment:

```
# Test production build locally
npm run build
npm run start
# Verify build succeeds without errors
```

## 2. Deploy to Vercel:

```
# Install Vercel CLI
npm install -g vercel
# Login to Vercel
vercel login
# Deploy
vercel
# Follow prompts and link to your project
```

## 3. Configure environment variables on Vercel:

- Go to Vercel Dashboard > Your Project > Settings > Environment Variables
- Add all variables from `.env.local`
- Redeploy if needed

## Backend Deployment (Railway/Render)

### 1. Prepare backend:

```
# Create requirements.txt if not exists
pip freeze > requirements.txt
# Create Procfile for Railway/Render
echo "web: uvicorn main:app --host 0.0.0.0 --port \${PORT}" > Procfile
```

### 2. Deploy to Railway:

- Connect GitHub repository
- Select backend directory
- Add environment variables from `backend/.env`
- Deploy

### 3. Update frontend API URL:

```
# In Vercel environment variables
NEXT_PUBLIC_API_URL=https://your-backend.railway.app
```

## Database Migration to Production

```
# Export local database
```

```
mysqldump -u root -p attireai_db > attireai_backup.sql

# Import to production database

mysql -h production-host -u user -p production_db < attireai_backup.sql

# Update DATABASE_URL in production environment
```

## Post-Deployment Checklist

- ☐ Test all authentication flows
  - ☐ Verify database connections
  - ☐ Check CORS configuration
  - ☐ Test API endpoints
  - ☐ Monitor error logs
  - ☐ Set up error tracking (Sentry recommended)
  - ☐ Configure CI/CD pipeline (optional)
- 

## Additional Resources

### Helpful Documentation Links

- **Next.js:** <https://nextjs.org/docs>
- **FastAPI:** <https://fastapi.tiangolo.com/>
- **Firebase Auth:** <https://firebase.google.com/docs/auth>
- **MySQL:** <https://dev.mysql.com/doc/>
- **Tailwind CSS:** <https://tailwindcss.com/docs>

### Team Communication

- **GitHub Issues:** For bug reports and feature requests
- **Team Meetings:** Tuesday, Thursday, Saturday 8:00 PM EST
- **Scrum Master:** Ekaterina Tsziao ([etszyao@purdue.edu](mailto:etszyao@purdue.edu))

### Getting Help

1. Check this documentation first
2. Search GitHub Issues for similar problems
3. Check framework documentation



4. Ask in team meetings
5. Contact team member responsible for that area:

- Authentication: Congtian Wu
  - Frontend/UI: Yuanfei Song
  - AI Recommendation Generation: Yichen Dai
  - Page Error: Ekaterina Tsziao
- 

**Last Updated:** March 9, 2026

**Document Version:** 1.0

**Maintained by:** Team 11